体系结构 exp13 实验报告

箱子：22

组员：叶驰骏 区梓俊 李皓

一、实验概述

本实验在 exp12 的 CSR 访问、syscall 异常和 ertn 返回机制基础上，继续完善 LoongArch 处理器的异常处理功能。exp13 主要加入取指地址错、访存地址不对齐、断点和指令不存在异常，同时扩展软件中断、硬件中断、定时器中断以及计数器读取通路。

处理器仍采用 IF、ID、EX、MEM、WB 五级流水线。每一级内部保存本阶段的流水寄存器，mycpu_top.v 主要负责阶段间连线和外部 SRAM 接口连接。异常在能够判断的流水级产生，再随指令传递到 WB，由 WB 统一保存异常状态、取消后续指令并重定向 PC。

exp12 和 exp13 都使用 csr_defs.vh 集中定义 CSR 编号和异常相关常量，并将 CSR 状态按字段维护。exp13 进一步修改各级异常接口，在 IF 增加未对齐取指请求的屏蔽，在 WB 统一进行异常优先级编码。原有补丁不能完整反映这次修订，以下以当前源码的接口和行为为准。

1.1 异常处理相关总线的组成

异常处理涉及两类总线：csr_bus 传递 CSR 访问、syscall 和 ertn 等控制信息，ex_bus 传递 exp13 新增的逐项异常标志及错误地址。它们都随指令逐级保存，但用途和位段不能混用。

exp12 的 csr_bus 为 82 位，ID 中的打包方式为：

```
id_ex_csr_bus <= {csr_re, csr_we, inst_ertn, inst_syscall,
                  inst[23:10],
                  (inst_csrwr ? 32'hffff_ffff : rj_value),
                  rkd_value};
```

位段	字段	作用
[81]	csr_re	CSR 读取标志，指示通用寄存器的写回结果来自 CSR 旧值；csrrd、csrwr、csrxchg 都置位
[80]	csr_we	CSR 写入标志，csrwr、csrxchg 置位；在 WB 结合 valid、复位和 flush 形成实际写使能
[79]	ertn	异常返回标志，在 WB 恢复模式并重定向到 ERA
[78]	syscall	系统调用标志，exp12 在 WB 根据它产生 SYS 异常
[77:64]	csr_num	14 位 CSR 编号，来自指令的 [23:10]，用于选择目标 CSR
[63:32]	csr_wmask	32 位写掩码；csrwr 为全 1，csrxchg 使用 rj 的值，位为 1 时更新对应位
[31:0]	csr_wdata	32 位写入数据；CSR 写指令使用 rd 原来的值，经第二读端口及前递得到 rkd_value

这 82 位由 4 个控制位、14 位编号和两组 32 位数据组成。csr_rdata 不在这条总线中，它是 CSR 模块在 WB 根据 csr_num 产生的读取结果。指令 PC、通用寄存器目的编号和流水有效位也通过各自的信号传递。

exp13 的 csr_bus 保留低 82 位布局，在高位增加两个控制标志，接口实际为 84 位：

新增位	字段	含义
[83]	rdcntvh	指令需要读取稳定计数器高 32 位
[82]	rdcntvl	指令需要读取稳定计数器低 32 位

exp13 的 ex_bus 为 38 位，布局为 `{flags[5:0], badv_or_pc[31:0]}`：

ex_bus 位段	对应标志	作用
[37]	flags[5]，INT	中断请求满足响应条件
[36]	flags[4]，ADEF	取指地址未按 4 字节对齐
[35]	flags[3]，ALE	数据访存地址不满足访问宽度的对齐要求
[34]	flags[2]，SYS	syscall 指令
[33]	flags[1]，BRK	break 指令
[32]	flags[0]，INE	指令编码不在支持的译码集合中
[31:0]	BADV/PC	ID 先保存指令 PC；EX 新增 ALE 时改为实际访存地址

这六位是独立异常标志，不是提前编码的 Ecode，也没有单独的 exception_valid 位。WB 对 [37:32] 做归约或，并结合 mem_wb_valid 判断是否提交异常，再按 INT、ADEF、ALE、SYS、BRK、INE 的顺序生成 Ecode。六位可以同时存在多个有效标志，因此还需要优先级选择。

exp12 仅靠 csr_bus 中的 syscall 位完成 SYS 异常识别；exp13 保留这个位段以维持原布局，但 WB 正式判断异常改用 ex_bus。两条总线均沿 ID/EX→EX/MEM→MEM/WB 传递，必须始终与同一条指令的 PC 和 valid 对齐。

1.2 CRMD 分域维护

exp12 和 exp13 分别保存当前特权级和中断允许位：

```
reg [1:0] crmd_plv;
reg       crmd_ie;
reg [1:0] prmd_pplv;
reg       prmd_pie;
```

CRMD 的各字段按本阶段实现范围处理如下：

位段	字段	当前维护方式
[1:0]	PLV	使用 crmd_plv 保存，表示当前特权级
[2]	IE	使用 crmd_ie 保存，表示全局中断允许

位段	字段	当前维护方式
[3]	DA	固定读 1，采用直接地址模式
[4]	PG	固定读 0，当前不实现分页模式
[6:5]	DATF	固定读 0
[8:7]	DATM	固定读 0
[31:9]	保留位	固定读 0，写入无效

读取时再将这些字段拼接为软件看到的 32 位 CSR：

```
`CSR_CRMD: csr_rdata = {28'b0, 1'b1, crmd_ie, crmd_plv};
`CSR_PRMD: csr_rdata = {29'b0, prmd_pie, prmd_pplv};
```

固定字段没有单独的可写存储，软件写入也不会改变 DA、PG、DATF、DATM。复位后 PLV=0、IE=0，因此 CRMD 读值为 32'h0000_0008。CSR 编号只是访问选择，实际的 PLV、IE 内容保存在 CSR 模块内部的寄存器中。

可写字段仍使用统一的掩码合并结果：

```
wire [31:0] masked_value = (csr_wmask & csr_wdata) |
                           (~csr_wmask & csr_rdata);
```

但更新 CRMD 时只取其中的 [1:0] 和 [2]，不会把整个 masked_value 写入 CRMD。各状态转换的优先级为复位、异常提交、ertn 返回、普通 CSR 写入：

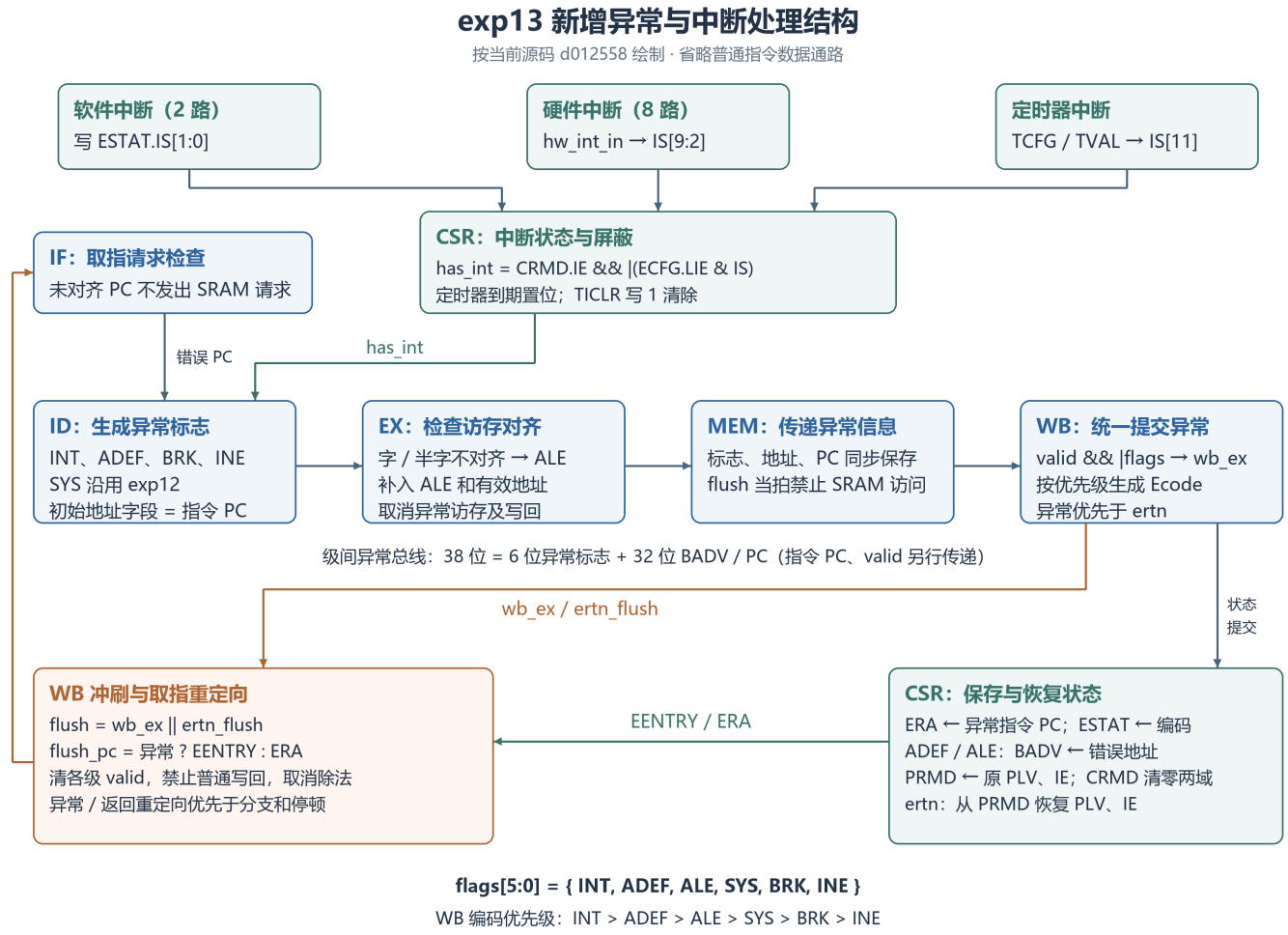
情况	crmd_plv	crmd_ie	PRMD 的处理
复位	清零	清零	prmd_pplv、prmd_pie 清零
wb_ex 有效	清零，进入 PLV0	清零，关闭全局中断	保存异常前的 PLV、IE
ertn_flush 有效	恢复 prmd_pplv	恢复 prmd_pie	保持原值
普通写 CRMD	更新为 masked_value[1:0]	更新为 masked_value[2]	保持原值

例如，发生异常前 PLV=3、IE=1，异常提交后 CRMD 的这两个字段变为 0，而 PRMD 保存 PPLV=3、PIE=1。CRMD 与 PRMD 都采用时钟沿上的非阻塞赋值，因此 PRMD 取得的是更新前的旧状态，不会把刚清零的值保存进去。PRMD 自身也允许通过 CSR 指令按掩码修改。

分域维护使每个字段的更新来源更明确：PLV、IE 由复位、异常、返回和 CSR 写入共同控制，固定模式位只参与读取拼接。ESTAT 也将软件中断位与异常编码分开维护，避免普通 CSR 写入改变硬件记录的异常类型。编号、Ecode 和异常标志常量集中放在 csr_defs.vh 中，便于对照各模块接口。

二、总体结构

流水级	职责	与其他部分的关系
IF 取指	取指令，管理 PC 和 IF/ID 寄存器，屏蔽未对齐取指请求	接收普通分支重定向；异常或 ertn 提交时优先接收 WB 的重定向地址
ID 译码	指令译码、读寄存器、相关检测，判断中断、ADEF、SYS、BRK 和 INE	接收 CSR 模块的中断状态，将各类异常标志与指令一起送入 EX
EX 执行	ALU 运算、乘除法、访存地址计算，判断 ALE	检查访存地址对齐，取消异常访存，向 MEM 传递异常信息
MEM 访存	访问数据 SRAM，产生字节写使能	保存异常信息；WB 冲刷时禁止后续指令访问 SRAM
WB 写回	选择写回结果，提交 CSR、异常和 ertn	更新异常状态，产生 flush 和 flush_pc，向 ID 反馈写回结果



主要模块及职责如下：

模块	功能
if_stage.v	PC 管理、指令 SRAM 请求、未对齐请求屏蔽、IF/ID 寄存器和取指重定向
id_stage.v	新增指令译码、异常判断、中断响应、CSR 相关阻塞和异常控制信息生成
ex_stage.v	访存地址对齐检查、异常访存取消、EX/MEM 异常信息保存

模块	功能
mem_stage.v	数据 SRAM 控制、MEM/WB 异常信息保存和 load、CSR 结果相关处理
wb_stage.v	异常优先级编码、统一提交、异常返回和 CSR 结果写回
csr.v	保存异常状态和中断配置，实现稳定计数器、定时器及相关 CSR
csr_defs.vh	集中定义 CSR 编号、基础 CSR 总线位号、异常标志位置和异常编码
mycpu_top.v	五个阶段的实例化，异常总线、硬件中断输入和 CSR 待提交状态的连接

三、exp13：其他异常处理

3.1 新增异常

exp12 已实现 syscall 异常，exp13 在此基础上加入 ADEF、ALE、BRK 和 INE，并将中断纳入同一条异常处理通路。

类型	Ecode	判断位置	产生条件
INT，中断	0x00	ID	全局中断允许，且存在未被屏蔽的中断请求
ADEF，取指地址错	0x08	ID	PC 不按 4 字节对齐，IF 同时屏蔽相应取指请求
ALE，地址不对齐	0x09	EX	字访问地址低两位不为 0，或半字访问地址最低位不为 0
SYS，系统调用	0x0b	ID	执行 syscall 指令，沿用 exp12 功能
BRK，断点	0x0c	ID	执行 break 指令
INE，指令不存在	0x0d	ID	指令未命中当前处理器支持的译码集合

本实验这些异常的 EsubCode 均记录为 0。各类异常先独立下传，WB 再按 INT、ADEF、ALE、SYS、BRK、INE 的优先级编码。EX 检测到 ALE 时先保留 ID 已经产生的异常，只有 ID 不带异常标志时才补入 ALE，避免后续判断覆盖原有异常信息。

3.2 ID 阶段的异常判断

ADEF 根据当前指令 PC 的对齐情况判断。LoongArch 指令长度为 4 字节，因此 PC[1:0] 应为 0，对应逻辑为：

```
wire adef = |if_id_pc[1:0];
```

IF 同时检查 next_pc[1:0]，不对未对齐的地址发起指令 SRAM 请求，但仍将错误 PC 传到 ID。ID 根据这个 PC 产生 ADEF 标志，再通过正常异常通路送到 WB。这样可以在阻止非法请求的同时保留准确的 ERA 和 BADV。

break 通过新增译码信号识别。INE 则使用已支持指令的译码结果构成白名单，所有合法运算、分支、CSR、异常返回和读计数器指令都应命中 inst_valid；未命中的指令产生 INE，不能按普通指令写入寄存器。

同时，pcaddu12i 的译码补充了对 inst[25] 的限制。原有译码条件过宽时，部分不属于该指令的编码可能被当成合法指令，加入 INE 后需要同步收紧这一条件。

ID 判断出异常后，仍将这条指令作为有效流水项向后传递，但将 ALU 操作、访存控制、通用寄存器写使能、CSR 写使能和 ertn 控制取消。分支重定向也要求当前指令没有异常。这样既保留了异常指令自身的 PC，又避免它继续执行原来的操作。

3.3 EX 阶段的地址对齐检查

访存有效地址由 EX 阶段的 ALU 计算，因此 ALE 在 EX 判断。不同宽度的要求如下：

- 字节访问没有额外的地址对齐限制；
- 半字访问要求 addr[0]=0；
- 字访问要求 addr[1:0]=00。

```
wire ale = (id_ex_res_from_mem || id_ex_mem_we) &&
           (((id_ex_mem_size == 2'b10) && (|alu_result[1:0])) ||
            ((id_ex_mem_size == 2'b01) && alu_result[0]));
```

检测到 ALE 后，EX 将实际访存地址作为 BADV 候选值保存，并清除送入 EX/MEM 的 load、store 和通用寄存器写使能。异常结果也不能通过 EX 前递给后续指令。

这里必须在访问 SRAM 之前取消操作。特别是 store，一旦在 MEM 写入存储器，后续 WB 即使产生异常，也不能仅靠清空流水寄存器恢复原来的内存内容。

3.4 异常信息的流水传递

exp13 使用第 1.1 节所列的 38 位异常总线，在 ID/EX、EX/MEM、MEM/WB 中逐级保存。ID 不提前生成 Ecode，而是独立生成六位标志：

```
wire [5:0] id_ex_flags = {has_int, adef, 1'b0, inst_syscall,
                          inst_break, !inst_valid};
wire id_exception = |id_ex_flags;
// ID/EX 寄存器更新时：
// id_ex_ex_bus <= {id_ex_flags, if_id_pc};
```

ALE 在 ID 尚不能判断，因此对应位置先填 0。EX 若发现 `|id_ex_ex_bus[37:32]` 为 1，则直接传递已有异常总线；否则在检测到 ALE 时，将 flags 置为 6'b001000，并将低 32 位换成 alu_result。没有 ALE 时继续保留原 PC。MEM 不重新判断异常，只负责同步保存。

异常总线必须与 valid、PC 和写回控制属于同一条指令。异常标志用于最终编码，低 32 位提供 BADV 候选值，而 ERA 使用另一路传递到 WB 的指令 PC；即使发生 ALE，也不能用数据访问地址代替异常返回地址。

3.5 WB 统一提交和异常返回

WB 根据 MEM/WB 的有效位和异常总线产生 wb_ex：

```
wire wb_ex = resetn && mem_wb_valid && (|mem_wb_ex_bus[37:32]);
wire ertn_flush = resetn && mem_wb_valid && ertn && !wb_ex;
```

```
assign flush = wb_ex || ertn_flush;
assign flush_pc = wb_ex ? ex_entry : ex_era;
```

WB 再根据六个标志生成 wb_ecode。全零标志不会触发 wb_ex；此时组合编码器末尾默认选择的 INE 编码也不会写入 ESTAT。wb_esubcode 当前固定为 9'b0，与 wb_ecode 一起送入 CSR 模块。

异常提交时，CSR 模块完成以下动作：

1. 将异常指令自身的 PC 保存到 ERA；
2. 将原 CRMD.PLV 和 CRMD.IE 保存到 PRMD；
3. 将当前 PLV 和 IE 清零，进入内核态并关闭全局中断；
4. 将 Ecode 和 EsubCode 写入 ESTAT；
5. 对 ADEF、ALE 更新 BADV，其他异常保持 BADV 原值；
6. 由 WB 发出冲刷信号，将 PC 重定向至 EENTRY。

ERA 不自动加 4。处理程序需要跳过异常指令时，应自行修改 ERA；中断返回时，则可以从被中断指令重新执行。执行 ertn 后，CRMD 的 PLV、IE 从 PRMD 恢复，PC 跳回 ERA。

WB 提交异常的同一周期禁止通用寄存器和 CSR 的普通写入，并通过 flush 取消更年轻的流水项。MEM 的 SRAM 使能也受 flush 限制，防止后续 store 在冲刷生效的时钟沿写入内存。这部分沿用 exp12 的冲刷结构，exp13 将新增异常接入统一入口。

四、exp13：中断、定时器和计数器

4.1 新增控制状态寄存器

CSR	编号	功能
ECFG	0x0004	LIE 域控制各路中断是否允许响应
BADV	0x0007	记录 ADEF、ALE 对应的错误地址；也允许软件按掩码写入，地址异常硬件更新优先
TID	0x0040	保存计数器编号，可由软件写入，由 rdcntid 读取
TCFG	0x0041	设置定时器初值、周期模式和使能
TVAL	0x0042	读取定时器当前值
TICLR	0x0044	有效写 1 清除定时器中断，读取结果为 0

CSR 写入继续使用 exp12 的掩码合并方式。csrwr 使用全 1 掩码，csrxchg 使用源寄存器提供的掩码，先将新旧值合并，再更新各寄存器允许写入的字段。

4.2 中断来源与屏蔽

本实验支持两路软件中断、八路硬件中断和一路定时器中断。它们在 ESTAT.IS 中的位置如下：

位段	中断来源	状态更新方式
[1:0]	软件中断	由软件写 ESTAT 设置或清除
[9:2]	八路硬件中断	每拍采样 hw_int_in[7:0]

位段	中断来源	状态更新方式
[10]	本实现固定为 0	不作为有效中断来源
[11]	定时器中断	定时器到期置位，通过 TICLR 清除
[12]	本实现固定为 0	未接入中断来源

中断请求只有同时满足全局允许和逐路允许，才送入 ID：

```
wire [12:0] interrupt_status =
    {1'b0, estat_timer, 1'b0, estat_hw, estat_sw};
assign has_int = crmd_ie && (|(ecfg_lie & interrupt_status));
```

CRMD.IE 控制全局中断，ECFG.LIE 控制各路中断。ECFG 写入时使用 13'h1bff 保留可写位，其中第 10 位固定为 0。中断被屏蔽时，状态仍然可以存在，只是不产生 has_int。

mycpu_top 新增 hw_int_in[7:0] 输入，最终连接至 CSR 模块。当前实验 SoC 没有外部中断源，因此在 soc_lite_top.v 中将它接为 8'b0，定向测试平台则逐路驱动输入。该接口要求输入属于 CPU 时钟域，异步外部中断需要在 SoC 中先完成同步。

4.3 CSR 修改与中断判断的顺序

中断允许位在 WB 更新，而 ID 比 WB 提前几个周期判断中断。如果前面的指令正在写 CRMD、ECFG 或 ESTAT，后面的指令直接使用旧状态判断，就可能多执行本应被中断取消的指令。

mycpu_top 将 EX、MEM、WB 中有效的 CSR 写指令合并为 csr_pending。只要存在较早的 CSR 写操作，ID 就暂时停止推进；等它提交后，再使用更新后的 has_int 判断当前指令是否需要响应中断。

```
assign ready_go = !flush && ex_allow_in && !csr_pending &&
                  !(resetn && if_id_valid && !id_exception &&
                    (hazard_rj || hazard_rkd));
```

这种处理没有增加 CSR 专用前递，而是通过停顿保证顺序。代价是即使前面的 CSR 写入与中断无关，ID 也会等待。当前实验优先保证功能正确，暂不细分每个 CSR 的相关关系。

4.4 定时器实现

TCFG 的 [31:2] 为 InitVal，bit1 为 Periodic，bit0 为 En。它们分别保存为 tcfg_initval、tcfg_periodic、tcfg_en，读取时再拼接为 tcfg_value。写 TCFG 时先完成掩码合并，再按域更新配置；合并后的 En=1 时，将 `{InitVal, 2'b0}` 装入 timer_value。

在 tcfg_en=1 且 timer_value 不为 32'hffff_ffff 时，倒计数器继续运行。在时钟沿检测到使能有效且当前值已经为 0 时，置位定时器中断，并根据工作模式决定后续动作：

- 单次模式从 0 减 1 得到 32'hffff_ffff，后续停止递减；
- 周期模式重新装入初值，继续下一轮计数。

软件通过 TVAL 读取当前定时器值；写 TCFG 关闭后，后续周期停止递减，但已经产生的中断状态不会自动消失。由于配置与计数器在不同时序块中更新，关闭 TCFG 的那个时钟沿仍可能按旧的使能值完成一次递减。

定时器中断状态保持到 TICLR 的 bit0 被有效写 1。对 csrxchg，还要求写掩码 bit0 为 1；写 0 不清除中断。到期和清除发生在同一拍时，本实现优先保留新的到期事件。

4.5 稳定计数器和读取指令

稳定计数器是独立的 64 位寄存器，复位时清零，此后每个时钟周期加 1。流水线停顿、异常处理以及定时器关闭都不影响它继续计数。它与 TVAL 对应的可编程定时器是两个独立的计数部件。

exp13 保留三条读取指令的译码，其目标功能如下：

指令	读取内容	写回目的寄存器
rdcntvl.w	稳定计数器低 32 位	rd
rdcntvh.w	稳定计数器高 32 位	rd
rdcntid	TID	编码中的 rj 字段

rdcntid 的 rj 字段表示目的寄存器，不是需要读取的源操作数。ID 将目的寄存器选为 rj，并将 CSR 编号固定为 0x0040，复用已有 CSR 读取通路。

原有 CSR 控制总线由 82 位扩展为 84 位。新增 bit83 表示读取计数器高位，bit82 表示读取低位，其余 [81:0] 保持 exp12 的布局。但当前版本的 WB 写回选择仅包含 csr_rdata、load_result 和 mem_wb_alu_result，已经没有根据 bit83、bit82 选择 stable_counter 的逻辑。因此稳定计数器本身正常递增、读指令标志正常下传，并不代表 rdcntvl.w、rdcntvh.w 能取得计数值；这两条指令的写回通路仍需补齐。rdcntid 则继续通过 csr_re 和 TID 读取通路写回。

4.6 读计数器指令的相关处理

EX 和 MEM 的 pending 条件检查 load、csr_bus[81] 和 csr_bus[80]，EX 还检查未完成除法；前递条件相应排除 load 和 CSR 访问。rdcntid 会设置 csr_re，因此后续指令使用它的结果时，仍会等待 WB，再通过同周期写回旁路取得 TID。

五、流水线接口和控制信号

阶段间的关键控制信号如下：

信号	作用
valid	标识流水级内容是否为真实指令，异常总线也必须结合有效位使用
ready_go	ID 当前指令是否满足下游接收、CSR 顺序和操作数相关条件
allow_in	EX 是否能够推进当前指令，多周期除法未完成时阻塞前端
id_ex_ex_bus / ex_mem_ex_bus / mem_wb_ex_bus	逐级传递 38 位异常信息，即六个异常标志和 32 位地址

信号	作用
id_ex_csr_bus / ex_mem_csr_bus / mem_wb_csr_bus	逐级传递 84 位 CSR、异常返回和计数器读取控制
has_int	全局允许和逐路屏蔽共同作用后的中断请求
crmd_plv / crmd_ie	CSR 内部独立保存当前特权级和全局中断允许位
csr_pending	表示前面仍有未提交的 CSR 写操作
wb_ex	WB 当前有效指令需要提交异常
flush / flush_pc	取消年轻指令，并指定异常入口或异常返回地址
ex_pending_* / mem_pending_*	标记尚不能提供最终结果的目的寄存器
rf_we / rf_wdata	WB 实际写回接口，同时用于 ID 同周期旁路

ready_go 仍为高电平有效，高表示 ID 可以推进，低表示需要等待。已经检测到异常的指令不再需要等待普通源操作数，但仍需遵守 EX 接收条件和较早 CSR 写入的顺序。

flush 的优先级高于普通停顿。即使前端因相关而等待，或 EX 正在执行除法，异常提交也必须能够清空对应有效位并重定向取指。除法取消逻辑继续使用 exp12 的实现，避免被冲刷指令留下的 complete 影响后续除法。

六、问题分析

6.1 异常不能只在 WB 禁止写回

只将 rf_we 拉低，能够阻止错误结果写入寄存器，却不能阻止 store 提前写入存储器。因此本实验在 ID 取消已知异常的执行控制，在 EX 取消地址不对齐的访存，并在 WB 冲刷时同步关闭 MEM 的 SRAM 访问。异常检测、访存取消和最终提交需要配合使用。

6.2 CSR 更新后的中断响应

当中断已经处于待处理状态，而一条 CSR 指令刚好打开 ECFG 中的允许位时，后续指令必须使用更新后的配置。若 ID 没有等待 CSR 提交，紧邻的 store 可能在中断真正进入流水线之前执行。csr_pending 将这类顺序关系显式加入阻塞条件，定向测试也专门检查了连续两条后续 store 都被取消的情况。

6.3 定时器到期与中断清除

定时器计数值与定时器中断状态分别保存。计数到期负责设置状态，TICLR 负责清除状态，关闭计数器不等于清除中断。将这两个过程混在一起，容易出现计数已经停止但中断被意外清除，或周期模式中断只持续一个周期的问题。

6.4 计数器写回与相关控制需要同步修改

新版仍有 rdcntvl.w、rdcntvh.w 的译码和总线标志，但 WB 已移除计数器结果选择，EX、MEM 也不再将这两类指令列为 pending。这个变化说明，检查指令是否实现不能只看译码，还需要沿目的寄存器、结果来源、写回选择和相关检测逐级核对。后续完善时，应将计数器高、低位写回和相关控制一起恢复，并加入读取后立即使用结果的测试。

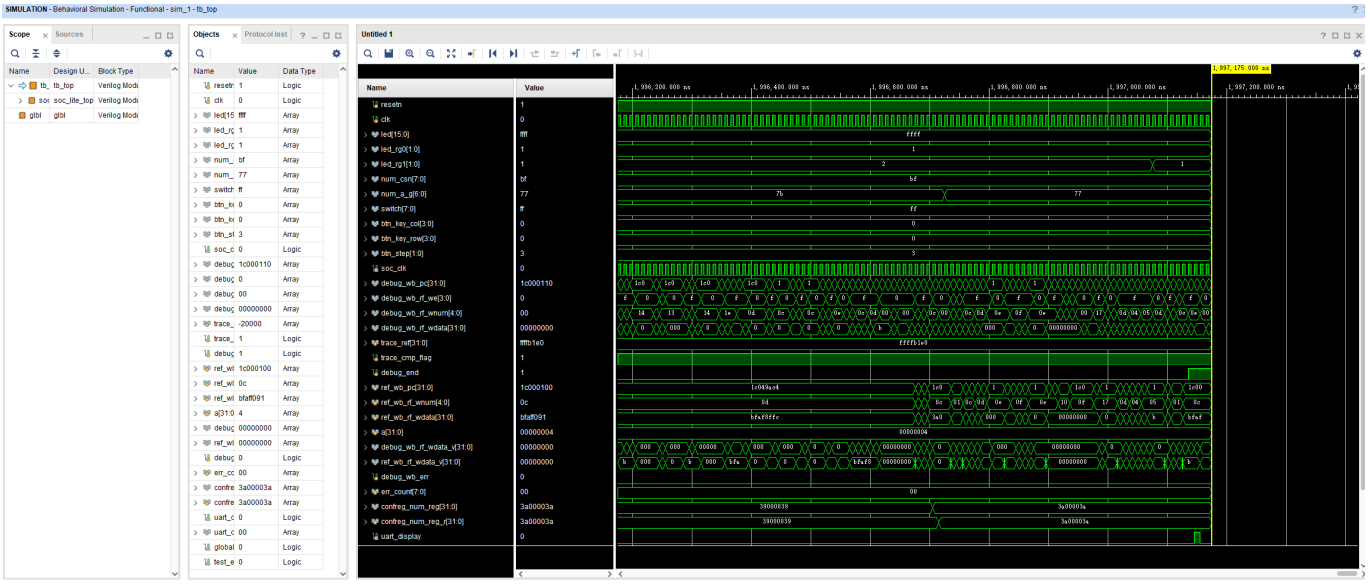
七、仿真与综合结果

7.1 功能测试

使用 Vivado 2019.2 进行编译、展开和仿真。根据 2026 年 10 月 25 日的验证记录，原始 func 测试与 golden_trace 比较通过，共完成 58 个功能测试点，最终输出如下：

```
----[1996715 ns] Number 8'd58 Functional Test Point PASS!!!
================================================================
Test end!
----PASS!!!
```

7.2 波形图



八、实验总结

exp13 在 exp12 的异常入口和返回机制基础上，补充了 ADEF、ALE、BRK、INE 以及多种中断来源。由 ID 和 EX 分别完成不同类型的异常判断，通过 38 位异常总线将六个独立标志和错误地址送入 WB，再统一编码、更新 ERA、ESTAT、PRMD、CRMD 和 BADV，并冲刷后续指令。

exp12 和 exp13 共同将 CRMD 按 PLV、IE 分域维护，固定模式位在读取时拼接，PRMD 分别保存和恢复原特权级与中断允许位。定时器也将配置字段与计数状态分开维护，利用 TCFG、TVAL、TICLR 完成定时与中断清除。

这次实验说明，异常支持不仅需要检测错误条件，还必须保证异常发生前的指令能够正常完成，异常指令及其后的指令不会产生错误副作用。异常信息、PC、写使能和中断配置的时序关系，是本次设计的重点。

九、调试过程与真实体会

这次实验最需要反复核对的是异常发生后，流水线里哪些操作已经完成，哪些操作还可以取消。通用寄存器在 WB 写入，而 store 在 MEM 就会访问 SRAM，因此不能只盯着最后的寄存器写使能。把异常指令和后续 store 的位置放在同一条时序线上，才能说明为什么 EX 要提前取消 ALE 访存，WB 冲刷时还要同步关闭 MEM 的访问。

中断与 CSR 写入之间的关系也比单独判断 has_int 更容易出错。一个中断请求可以一直保持，但前面的 ECFG 或 CRMD 写入会改变它是否允许响应。定向测试专门检查了打开中断后紧跟两条 store 的情况，关注的是内存

有没有实际被写入，而不仅是最后有没有跳进异常处理程序。

计数器部分需要分清“正在计数”和“中断已经发生”两件事。稳定计数器每拍运行，定时器由 TCFG 控制，定时器中断则由独立状态位保持。软件关闭定时器之后，仍可能需要写 TICLR 清除已经产生的请求。将三个状态分别考虑，逻辑和波形都会更清楚。

这次核对代码时，异常信息由提前编码改成了逐项标志，CRMD 也由整体状态改成了按域保存。阅读波形时，应分别观察六个异常标志和最终 Ecode，以及 crmd_plv、crmd_ie 与 prmd_pplv、prmd_pie 的对应变化。这样既能看出异常从哪里产生，也能看出异常进入和返回时究竟修改了哪些状态。